

LAB 4

Processes and the exec system call

DEADLINE: 05/14/2026

Lab Overview:

For this lab, we will be learning how to create a controlled number of processes and use them to execute other programs.

Warning: This lab could potentially crash your VM, forcing a restart. Do not test more than 7 processes until 100% certain.

Core Tasks:

1. Create a child process.
2. Call `exec` to execute another program.
3. Create multiple processes to execute more programs.
4. Use existing code to monitor the current running processes.

Task Details:

1. Create a process.
 - a. Use **`fork()`** to create a child process.
2. Call `exec` to execute another program.
 - a. Use **`execvp()`** to execute **`“./iobound -seconds 5”`**.
 - b. Wait for the child process to finish.
3. Create multiple processes to execute more programs.
 - a. Ask the user from the command line for the number of processes(**`execute with ./lab.out n`**) where `n` is the number of processes.
 - b. Create an array of **`pid_t`**
 - c. Utilize a loop to start the exact number of processes.
 - d. Wait for all the children processes to finish.
4. Open another terminal and use the command `“top”` to check the number of processes you created. Note, the processes' names are **`“./iobound”`**
5. Once you are certain that you can spawn the correct number of processes, you may use

existing code to monitor the current running processes as you intended to start for the rest of this project.

a. Use the following function in the code provided to generate the monitor script

```
void script_print (pid_t* pid_ary, int size)
```

NOTE: This function does not work on IX-Dev; you will receive the following error:

```
top command:: No such file or directory.
```

Ignore this error. Use step 4 (another terminal connected to IX-Dev) for monitoring processes.

You should still include this function as described in your code for testing purposes.

NOTE: if you are running a VM then this function may or may not work depending on your setup. Do NOT do this step if you do not need to. If it does not work, try running "unset GTK_PATH" in a VS-code terminal.

Submission Requirements:

Task 1&2 are practice steps, the final products are the result of step 3&4.

Here is what you have to submit:

1. lab4.c
2. iobound.c
3. Makefile
4. log.txt
5. Screenshot of compiling and system information.

The naming convention of the zip/tar file while uploading to Canvas

UoID_duckID_LabX (an example is given below)

- First Name : Dewi
- Last Name: Yokelson
- Submission for: Lab4
- So the name of the zip/tar file should be:

Dewi_Yokelson_Lab4.zip